

Project Reflection: Agentic Incident Flow

The hardest part:

Surprisingly, the hardest part wasn't the AI logic; it was the environment and authentication plumbing around it.

I started on Python 3.14, which turned out to be too new for some of the required libraries. pydantic needed to compile a Rust component from source, which failed because I didn't have a C++ linker installed. The fix was installing Python 3.11 alongside 3.14 and creating a virtual environment pinned to that version. A good lesson that "newest is best" isn't always true for dependency compatibility.

The second, and honestly bigger, challenge was ServiceNow authentication. My webhook was receiving tickets correctly and Gemini was returning decisions, but every write-back to ServiceNow failed with a 401 Unauthorized even though I could log into ServiceNow UI with the exact same username and password. I had to methodically rule out causes one at a time: PowerShell quoting issues with special characters in the password, a malformed instance URL, and finally a newer ServiceNow security feature (Basic Auth Restrictions / MFA enforcement) that blocks API-level Basic Auth even when credentials are completely valid. Debugging this required testing the raw HTTP request outside of my own code to prove the problem wasn't in my Python; it was an instance-level setting I hadn't known existed.

What would I improve with more time?

Move off Basic Auth entirely: I'd switch to an OAuth token for the ServiceNow integration instead of a username/password pair — it's the more secure, production-appropriate approach and avoids the exact authentication issue I hit.

Add a proper test suite: Right now correctness is verified by manually creating tickets and watching logs. I'd add automated tests that mock the Gemini and ServiceNow calls, so the webhook logic and duplicate-protection logic can be verified without needing a live PDI or API key.

Persist the duplicate-processing guard: The in-memory set works for this task, but it resets every time the service restarts. A lightweight database (even SQLite) would make it durable across restarts.

Handle the ngrok URL rotation problem: Every time I restart ngrok on the free tier, the public URL changes, and I have to manually update the Business Rule. With more time or a paid ngrok plan, I'd use a static domain, or add a small script that updates the Business Rule automatically via the ServiceNow API when the tunnel restarts.